

Kalman Meets Curriculum: Efficient Dynamic Prompt Selection for Adaptive RL Finetuning

Haodong Zhu^{1,2*} Yangyang Ren^{1,2*} Yanjing Li^{3†} Sheng Xu^{4†}
Haiguang Liu² Linlin Yang³ Baochang Zhang^{1‡}

¹Beihang University ²Zhongguancun Academy ³Nanyang Technological university

⁴Independent Researcher ⁵Communication University of China

HaodongZhu@buaa.edu.cn yyren@buaa.edu.cn

Abstract

Reinforcement learning (RL) finetuning significantly enhances the reasoning capabilities of large language models (LLMs), yet its effectiveness critically depends on selecting prompts of appropriate difficulty for the current policy. This is challenging because prompt difficulty evolves throughout training. Existing online methods therefore face a trade-off: evaluation-based approaches are accurate but expensive, while prediction-based approaches are efficient but typically assume stationary difficulty, making them ill-suited to RL’s non-stationary training dynamics. To address these issues, we propose a *Kalman-Guided Prompt Selection* method (*KGPS*), which reformulates prompt selection as a dynamic state estimation problem rather than static difficulty prediction. KGPS models each prompt’s latent success rate in logit space using a linear-Gaussian state-space model, with process noise coupled to the magnitude of policy updates so that uncertainty increases when the policy changes more substantially. A Kalman filter then maintains a calibrated Gaussian posterior over prompt difficulty, and prompts are selected by maximizing a posterior-expected training utility that favors intermediate-difficulty prompts while naturally revisiting uncertain ones. The resulting procedure is adaptive to policy drift and requires no additional rollouts beyond standard policy training. Extensive experiments across mathematics, planning, and geometry reasoning benchmarks, as well as multiple RL algorithms, show that KGPS consistently improves both final accuracy and rollout efficiency over strong baselines, establishing state-of-the-art performance among online prompt selection methods. For example, on DeepSeek-R1-Distill-7B, KGPS uses 83% fewer rollouts than DS while even improving the average performance by 0.12 point across six math reasoning benchmarks.

1 Introduction

Reinforcement learning (RL) has become a cornerstone technique for post-training Large Language Models (LLMs), driving remarkable advances in both instruction following and complex reasoning [Shao et al., 2024, DeepSeek-AI et al., 2025, Zeng et al., 2025, Luo et al., 2025]. However, realizing its full potential remains non-trivial. RL optimization is known to be highly sensitive to training sample selection [Parashar et al., 2025, Qu et al., 2025, Shen et al., 2025a,b], because prompts that are already mastered by the current policy contribute little learning signal, while prompts that remain far beyond the policy’s capability often produce uninformative gradients. In practice, the most

*Equal contribution.

†Corresponding author.

‡project lead.

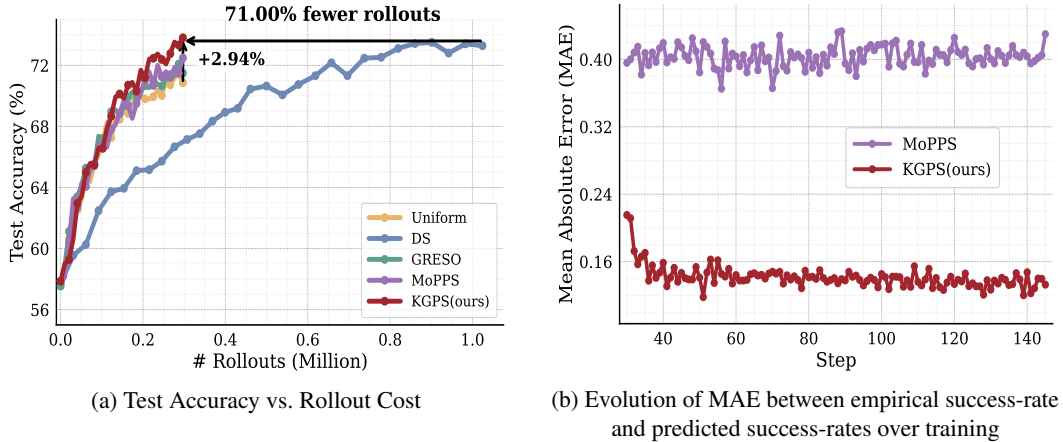


Figure 1: On the Math dataset using the Qwen3-0.6B model, (a) KGPS obtains comparable performance to evaluation-based DS (Oracle) with 71.00% fewer rollouts, also surpasses vanilla uniform method by up to +2.94% higher test accuracy with the same rollout cost. Meanwhile, compared to prediction-based prompt selection methods GRESO and MoPPS, the proposed KGPS achieves up to +2.33% higher test accuracy. (b) We calculate the MAE between empirical success-rate and predicted success-rate, where KGPS maintains substantially lower MAE than MoPPS throughout training, demonstrating superior prompt difficulty prediction without additional LLM inference.

useful prompts tend to be those at an appropriate difficulty for the current policy. Crucially, prompt difficulty is not fixed: as the policy improves, previously informative prompts may become trivial, and the set of useful training samples correspondingly shifts over time. This dynamic undermines static selection strategies—including uniform sampling and fixed curricula—which can waste rollout budget on uninformative prompts and become increasingly misaligned with the model’s evolving competence [Yu et al., 2025, Bae et al., 2025]. This underscores the need for adaptive task selection that remains aligned with the current policy throughout training.

Prior work on data selection for RL training splits into offline and online approaches. Offline methods precompute static difficulty estimates and use fixed curricula [Parashar et al., 2025, Wen et al., 2025, Shen et al., 2025b], but these misalign as the policy improves. Online methods dynamically recompute sample utility using real-time feedback [Qu et al., 2025, Shen et al., 2025a, Yu et al., 2025], maintaining alignment with evolving competence. In RL finetuning, this online setting is especially important because prompt success rate under the current policy is itself non-stationary.

Online selection methods broadly fall into two categories depending on how sample utility is estimated. Evaluation-based approaches, such as Dynamic Sampling (DS) [Yu et al., 2025], operate by oversampling a large candidate set, executing rollouts across all candidates, and filtering prompts based on the observed reward outcomes. While yielding reliable utility estimates, these candidate evaluations demand substantial additional rollouts beyond those already needed for policy training, resulting in significant inference overhead that compounds at every update step. As illustrated in Fig. 1 (a), in DS training, this overhead (blue curve) can reach up to $7\times$ more rollouts than standard GRPO training (yellow curve), severely limiting practical scalability. To address this, prediction-based approaches, notably MoPPS [Qu et al., 2025] (purple curve) and GRESO [Zheng et al., 2025] (green curve), rely on lightweight proxy signals to identify prompts near a target difficulty level without requiring additional rollouts for selection. However, they model each prompt’s success rate as a stationary latent parameter, relying on a heuristic decay factor as a substitute for principled non-stationarity handling, which may yield inaccurate difficulty estimates as the policy evolves. To quantify this estimation inaccuracy, we calculate the MAE between predicted success-rate (before rollouts) and empirical success-rate (after rollouts) at every training step in Fig. 1 (b). As can be seen, this static modeling Qu et al. [2025] assumption results in persistently high estimation error throughout training ($\text{MAE} \approx 0.40$ in purple curve), failing to improve as the policy evolves. **These limitations call for a principled shift from static parameter estimation to dynamic state tracking**, one that preserves the efficiency of inference-free selection while better matching the non-stationary nature of RL finetuning.

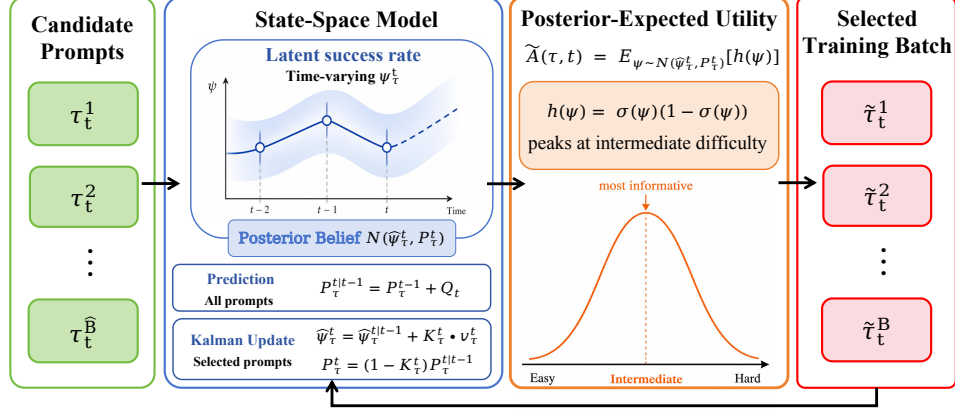


Figure 2: **Framework Overview of KGPS.** At each training step t , KGPS maintains a Gaussian posterior $\mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)$ over the latent logit-space difficulty of every candidate prompt via a Kalman filter. Prompts selected in the previous step receive a full Kalman update from their rollout observations; unselected prompts undergo only the prediction step, with variance inflated by Q_t to reflect policy-induced uncertainty growth. The updated posteriors drive prompt selection through the posterior-expected utility $\tilde{A}(\tau, t) = \mathbb{E}_{\psi \sim \mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)}[h(\psi)]$, where $h(\psi) = \sigma(\psi)(1 - \sigma(\psi))$ peaks at intermediate difficulty, ensuring the selected batch targets the most informative training signal while naturally revisiting long-neglected prompts.

To address this issue, in this paper, we propose *Kalman-Guided Prompt Selection (KGPS)*, a principled framework for online prompt selection that reframes difficulty tracking as a dynamic state estimation problem, as illustrated in Fig. 2. Rather than modeling each prompt’s success rate as a stationary latent parameter, KGPS adopts a linear-Gaussian state-space formulation in logit space, where the latent success rate of each prompt evolves as a random walk whose process noise is explicitly coupled to the magnitude of the policy update at each training step. This design ensures that the posterior uncertainty of every prompt, including those not selected for training, is continuously inflated to reflect the current state of the policy. Building on this formulation, we derive a Kalman filter that maintains a Gaussian belief over each prompt’s logit-transformed success rate, propagating uncertainty in a theoretically grounded manner. Prompt selection is then performed by maximizing a posterior-expected training utility score that prioritizes prompts at intermediate difficulty under the current policy, while naturally promoting exploration of prompts with accumulated uncertainty via posterior variance inflation. Importantly, KGPS operates on the same rollout outcomes already generated for policy training and requires no additional rollouts for prompt selection. As illustrated in Fig. 1, KGPS substantially reduces success rate estimation error (red curve in Fig. 1 (b)) and achieves superior performance under an equivalent rollout budget (red curve in Fig. 1 (a)) compared to MoPPS [Qu et al., 2025], a representative prediction-based method that relies on static posterior approximation, empirically validating the advantage of dynamic state estimation. The primary contributions of this work are three-fold:

- We reformulate prompt difficulty tracking as a dynamic state estimation problem, modeling each prompt’s latent success rate via a linear-Gaussian state-space model with process noise coupled to the policy update magnitude.
- We derive a Kalman filter for online difficulty tracking and propose a posterior-expected training utility score that naturally balances exploitation of current estimates with exploration of uncertain prompts, requiring no hand-tuned decay schedule.
- Extensive experiments across diverse reasoning tasks and RL algorithms demonstrate that KGPS consistently outperforms existing baselines in both accuracy and rollout efficiency with negligible computational overhead.

2 Related Work

RL Finetuning of LLMs. Reinforcement learning has become a central paradigm for improving large language models, from alignment-oriented RLHF [Dong et al., 2024, Dai et al., 2023, Zheng

et al., 2023] to reasoning-oriented RLVR [Jaech et al., 2024, Guo et al., 2025, Team et al., 2025, Pan et al., 2025]. PPO [Schulman et al., 2017] remains a standard backbone. In the meantime, GRPO [Shao et al., 2024, DeepSeek-AI et al., 2025] offers a lighter alternative. Recent work further improves RL finetuning in terms of stability, sample efficiency, and scalability [Yu et al., 2025, Hu, 2025, Yue et al., 2025, Kazemnejad et al., 2024, Sheng et al., 2024].

Prompt Selection for RL Finetuning. Data curation has emerged as a promising strategy for improving RL finetuning efficiency. *Offline* methods select prompts prior to training based on static criteria such as difficulty or diversity [Ye et al., 2025, Li et al., 2025, Hu et al., 2025, Yang et al., 2024, Fatemi et al., 2025], but cannot adapt to the model’s evolving capabilities. *Online* methods address this by dynamically curating training batches, either filtering prompts with degenerate success rates [Yu et al., 2025, Liu et al., 2025, Cui et al., 2025, Meng et al., 2025] or prioritizing intermediate-difficulty prompts to maximize gradient informativeness [Bae et al., 2025, Chen et al., 2025], yet typically require additional rollouts over a large candidate pool, introducing substantial computational overhead. MoPPS [Qu et al., 2025] mitigates this cost via inference-free prompt selection using Beta–Bernoulli posteriors with Thompson Sampling, but its static formulation assumes time-invariant prompt difficulty, failing to account for the non-stationary evolution induced by a continuously improving policy.

Our work is most closely related to these online prediction-based methods, but **differs** in one key aspect: instead of static parameter estimation with heuristic forgetting, we formulate prompt difficulty tracking as a dynamic state estimation problem. This allows uncertainty to evolve with policy updates and yields more accurate prompt difficulty estimates without requiring additional rollouts.

3 Methodology

This section presents **KGPS (Kalman-Guided Prompt Selection)**, as shown in Figure 2, our framework for addressing the non-stationarity problem identified in Sec. 3.1. The key idea is to replace static difficulty estimation with dynamic state tracking. Concretely, KGPS proceeds in three stages: (i) modeling each prompt’s time-varying success rate as a latent state in a linear-Gaussian state-space model (Sec. 3.2); (ii) applying a Kalman filter to propagate a Gaussian posterior over every prompt’s logit success rate as the policy evolves (Sec. 3.3); and (iii) selecting prompts by maximizing a posterior-expected training utility score derived from this posterior (Sec. 3.4).

3.1 Preliminary

Setup and notation. Let $\mathcal{T} = \{\tau_i\}_{i=1}^N$ denote the prompt pool of size N , π_{θ_t} is the policy at training step t , and $\mathcal{T}_B^t \subset \mathcal{T}$ denotes the selected batch of size B . For each selected prompt $\tau \in \mathcal{T}_B^t$, the policy generates k independent rollouts, each scored via a binary reward following a Bernoulli distribution [Qu et al., 2025]:

$$r_{\tau}^{t,j} \sim \text{Bernoulli}(\phi_{\tau}^t), \quad j = 1, \dots, k, \quad (1)$$

where $\phi_{\tau}^t \in [0, 1]$ is the *latent success rate* of prompt τ under π_{θ_t} , serving as a surrogate for its difficulty. The empirical success count is $s_{\tau}^t = \sum_{j=1}^k r_{\tau}^{t,j}$, so that $s_{\tau}^t | \phi_{\tau}^t \sim \text{Binomial}(k, \phi_{\tau}^t)$.

Bayesian online prompt selection. Existing methods [Qu et al., 2025] model each prompt as an arm in a Bernoulli bandit and maintain a Beta–Bernoulli conjugate posterior over ϕ_{τ}^t . Initialized with a uniform prior $\phi_{\tau}^0 \sim \text{Beta}(1, 1)$, the posterior admits a closed-form recursive update:

$$\phi_{\tau}^t | \mathcal{H}_t \sim \text{Beta}(\alpha_{\tau}^t, \beta_{\tau}^t), \quad \alpha_{\tau}^{t+1} = \lambda \alpha_{\tau}^t + s_{\tau}^t, \quad \beta_{\tau}^{t+1} = \lambda \beta_{\tau}^t + k - s_{\tau}^t, \quad (2)$$

where $\mathcal{H}_t = \{\mathcal{T}_i^B, \mathcal{R}_i^B\}_{i=0}^t$ is the optimization history and $\lambda \in (0, 1)$ is a decay factor introduced to discount stale observations. Prompts are then selected by preferring posterior means $\hat{\phi}_{\tau}^t = \alpha_{\tau}^t / (\alpha_{\tau}^t + \beta_{\tau}^t)$ near the target $\phi^* \approx 0.5$ [Bae et al., 2025, Chen et al., 2025], where gradient informativeness is highest.

Limitation: the non-stationarity problem. The Beta–Bernoulli framework implicitly treats ϕ_{τ}^t as a stationary latent parameter; while the decay factor λ down-weights stale observations, it provides no principled mechanism for predicting how ϕ_{τ}^t evolves between selections. As shown in Figure 1 (b), this leads to persistently high estimation error throughout training. In early training, the Beta

posterior remains diffuse despite warmup initialization. As MoPPS [Qu et al., 2025] acknowledge, estimation accuracy improves only as the pseudo-count $C_\tau^t = \alpha_\tau^t + \beta_\tau^t$ grows, a condition that is rarely met given the limited rollouts available per prompt in practice. Thompson Sampling further compounds the issue by introducing additional stochastic variance into the predicted success rates. In later training, the fixed λ discounts observations at a constant rate regardless of policy update magnitude, failing to track the non-stationarity difficulty drift induced by a continuously improving policy π_{θ_t} . Resolving this model–reality mismatch requires reframing prompt difficulty tracking from static parameter estimation to *dynamic state estimation*, which is the central objective of KGPS.

3.2 State-Space Formulation in Logit Space

We treat each prompt’s success rate as a **time-varying latent state** and adopt a linear-Gaussian state-space model (SSM) to track it. This choice is motivated by two considerations: (i) SSMs provide a principled, closed-form mechanism for propagating uncertainty under non-stationarity, without recourse to hand-tuned decay schedules; and (ii) linear-Gaussian SSMs admit the Kalman filter under Gaussian approximation, producing a well-calibrated posterior that Sec. 3.4 subsequently uses to score and select prompts. Let $\phi_\tau^t \in [0, 1]$ denote the expected success rate of prompt τ under the current policy π_{θ_t} at step t . We work in logit space with $\psi_\tau^t = \text{logit}(\phi_\tau^t) \in (-\infty, +\infty)$ as the latent state, satisfying the unconstrained domain requirement of the linear-Gaussian framework.

State evolution. We model the latent logit ψ_τ^t as a random walk:

$$\psi_\tau^t = \psi_\tau^{t-1} + w_\tau^t, \quad w_\tau^t \sim \mathcal{N}(0, Q_t), \quad (3)$$

where the process noise variance is tied to the magnitude of the preceding policy update:

$$Q_t = \gamma \cdot \|\theta_t - \theta_{t-1}\|_2, \quad (4)$$

with $\gamma > 0$ a scalar hyperparameter. This coupling reflects a natural intuition: a larger policy step is likely to shift prompt difficulty more, warranting greater state uncertainty. Note that Q_t depends only on the *previous* parameter update, which is available at the start of step t before any new rollouts are collected. This ensures the prediction step (Sec. 3.3) is executable without circular dependency.

Observation model. At step t , if prompt τ is selected with k rollouts yielding s_τ^t successes, the empirical success rate $\hat{\phi}_\tau^t = s_\tau^t/k$ follows a scaled Binomial distribution (as defined in Sec. 3.1), which is incompatible with the Gaussian observation requirement of the linear-Gaussian framework. We therefore apply the delta method to the logit-transformed $\hat{\phi}_\tau^t$, which propagates the known variance $\text{Var}(\hat{\phi}_\tau^t) = \phi_\tau^t(1 - \phi_\tau^t)/k$ through the logit transform to yield a tractable Gaussian approximation in logit space (derivation in Appendix C):

$$\text{logit}(\hat{\phi}_\tau^t) = \psi_\tau^t + \varepsilon_\tau^t, \quad \varepsilon_\tau^t \sim \mathcal{N}(0, R_\tau^t). \quad (5)$$

Since the observation noise variance $R_\tau^t = 1/[k\phi_\tau^t(1 - \phi_\tau^t)]$ depends on the unknown ϕ_τ^t , we substitute the current posterior mean $\hat{\psi}_\tau^t$ at the start of step t before rollouts are collected, produced by the Kalman filter at step $t - 1$ (see Sec. 3.3). $\hat{\psi}_\tau^t$ is in fact a plug-in estimator that models the empirical success rate $\hat{\phi}_\tau^t$ as:

$$\tilde{\phi}_\tau^t = \text{clip}(\sigma(\hat{\psi}_\tau^t), \delta, 1 - \delta), \quad (6)$$

where $\delta = 1/(2k)$ ensures R_τ^t remains bounded above, and $\sigma(\cdot)$ denotes the sigmoid function mapping from logit space back to $[0, 1]$. Therefore, the observation noise variance is formulated as:

$$R_\tau^t = \frac{1}{k \tilde{\phi}_\tau^t (1 - \tilde{\phi}_\tau^t)}. \quad (7)$$

3.3 Kalman Filtering for Posterior Maintenance

With the SSM of Eqs. (3)–(7) in hand, we apply a Kalman filter (KF) to maintain a Gaussian posterior $\mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)$ over the latent logit state ψ_τ^t for every prompt $\tau \in \mathcal{T}$, where $\hat{\psi}_\tau^t$ is the posterior mean estimate and P_τ^t is the posterior variance. The filter’s responsibility is to maintain well-calibrated

posteriors as the policy evolves, with the resulting pair $(\hat{\psi}_\tau^t, P_\tau^t)$ subsequently serving as the input to the training utility score and prompt selection in Sec. 3.4. Since both the state transition and observation maps are linear with identity coefficients, the KF reduces to simple scalar recursions.

Prediction step (all prompts, every step). At each training step t , we inflate the posterior variance of *all* prompts to account for the latest policy update, regardless of whether they were selected:

$$P_\tau^{t|t-1} = P_\tau^{t-1} + Q_t, \quad \forall \tau \in \mathcal{T}. \quad (8)$$

Because $Q_t > 0$, the predicted variance of every prompt grows at each step, even for those not recently evaluated. This monotonic growth is what allows the learnability score in Sec. 3.4 to naturally favour prompts that have been neglected: their inflated $P_\tau^{t|t-1}$ raises their score deterministically until they are eventually selected.

Update step (selected prompts only). For each $\tau \in \mathcal{T}_{t-1}^B$, *i.e.*, prompts selected at step $t-1$, the filter incorporates the logit-space observation via the standard KF equations:

$$K_\tau^t = \frac{P_\tau^{t|t-1}}{P_\tau^{t|t-1} + R_\tau^{t-1}}, \quad \hat{\psi}_\tau^t = \hat{\psi}_\tau^{t|t-1} + K_\tau^t \cdot \nu_\tau^t, \quad P_\tau^t = (1 - K_\tau^t) P_\tau^{t|t-1}. \quad (9)$$

where $\hat{\psi}_\tau^{t|t-1} = \hat{\psi}_\tau^{t-1}$ is the one-step predicted mean and $\nu_\tau^t = \text{logit}(\hat{\phi}_\tau^{t-1}) - \hat{\psi}_\tau^{t|t-1}$ is the innovation. For prompts not selected at step $t-1$, the state estimate is unchanged and the variance retains its predicted value $P_\tau^{t|t-1}$. The Kalman gain $K_\tau^t \in (0, 1]$ adaptively weights the new observation against the prior belief. When the rollout is near-degenerate, the clipping in Eq. (7) ensures R_τ^{t-1} remains bounded, so the filter retains a non-negligible update even for extreme observations. Conversely, when the prior is diffuse ($P_\tau^{t|t-1}$ large) and the observation is reliable (moderate $\hat{\phi}_\tau^{t-1}$, small R_τ^{t-1}), $K_\tau^t \rightarrow 1$ and the posterior mean shifts aggressively toward the new data. In both cases, the updated $(\hat{\psi}_\tau^t, P_\tau^t)$ faithfully encodes the filter’s current state of knowledge about each prompt’s difficulty, which is precisely what Sec. 3.4 requires.

3.4 Prompt Selection via Posterior-Expected Utility

At each training step t , the Kalman filter delivers a posterior $\mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)$ for each prompt. We now show how to convert this posterior into a scalar training utility score and use it to select the most informative batch for the next training step.

From point estimates to posterior expectations. Following prior work [Qu et al., 2025, Bae et al., 2025, Chen et al., 2025], training efficiency is maximized by selecting prompts at intermediate difficulty, where the current policy is neither saturated nor overwhelmed. This motivates scoring each prompt by its estimated training utility $h(\hat{\psi}_\tau^t)$, where $h(\psi) = \sigma(\psi)(1 - \sigma(\psi))$ peaks at intermediate success rates and vanishes at the extremes. This criterion, however, ignores the posterior variance P_τ^t : a prompt with a near-degenerate success rate is persistently excluded from future selection even when its true difficulty under the current policy remains unknown. We resolve this by replacing the point evaluation with its expectation under the full Gaussian posterior maintained by the filter:

$$\tilde{A}(\tau, t) = \mathbb{E}_{\psi \sim \mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)}[h(\psi)]. \quad (10)$$

Because $h(\psi)$ is nonlinear and bounded, $\tilde{A}(\tau, t)$ is greater than $h(\hat{\psi}_\tau^t)$ whenever $P_\tau^t > 0$: integrating over the posterior spreads probability mass toward the intermediate-difficulty region where h is large, so any prompt with accumulated uncertainty retains a non-negligible score regardless of where its mean currently sits. Crucially, no stochastic sampling is needed: the prediction step in Eq. (8) continuously inflates P_τ^t for unobserved prompts, which raises $\tilde{A}(\tau, t)$ deterministically until the prompt is revisited.

Efficient computation via Gauss–Hermite quadrature. The expectation in Eq. (10) has no closed form. Since $h(\psi)$ is a smooth, bounded function of a scalar Gaussian variable, we evaluate it efficiently

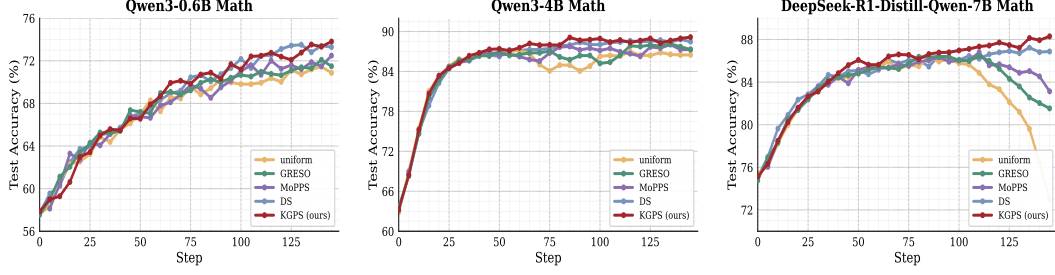


Figure 3: Test accuracy on the Math benchmark across three model scales under different data selection strategies. KGPS (ours) consistently achieves top performance.

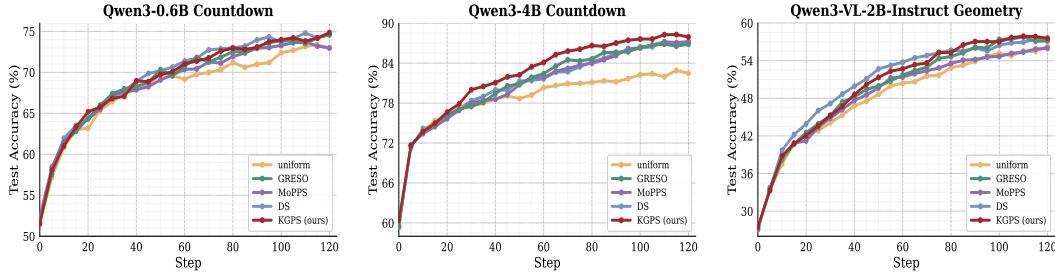


Figure 4: Test accuracy on Countdown and Geometry benchmarks, where KGPS (ours) demonstrates consistent advantages over all baseline data selection benchmarks across varying model scales.

via five-point Gauss–Hermite quadrature (detailed in Appendix D). Substituting $\psi = \hat{\psi}_\tau^t + \sqrt{2P_\tau^t}x$ yields:

$$\tilde{A}(\tau, t) \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^5 w_i h\left(\hat{\psi}_\tau^t + \sqrt{2P_\tau^t}x_i\right), \quad (11)$$

where the node–weight pairs $(x_i, w_i) \in \{(0, 0.9453), (\pm 0.9586, 0.3936), (\pm 2.0202, 0.0200)\}$ are fixed constants requiring no tuning. The per-prompt cost is five scalar evaluations of $h(\psi)$, negligible relative to LLM rollout overhead.

Prompt selection. At each step t , we compute $\tilde{A}(\tau, t)$ for every $\tau \in \mathcal{T}$ and select the training batch by greedy maximization:

$$\mathcal{T}_B^t = \text{Top-}B\{\tau \in \mathcal{T} \mid \tilde{A}(\tau, t)\}. \quad (12)$$

Before selection begins, we run a single warmup epoch in which prompts are sampled uniformly to obtain an initial empirical success rate $\hat{\phi}_\tau^0$ for every $\tau \in \mathcal{T}$. The Kalman filter is then initialized with $\hat{\psi}_\tau^0 = \text{logit}(\hat{\phi}_\tau^0)$, providing a data-driven starting point for the posterior mean rather than a uninformative init prior. The full KGPS pipeline is summarized in Algorithm 1 in the Appendix.

4 Experiment

4.1 Experimental Setup

Datasets and Models. We evaluate KGPS on three reasoning benchmarks across distinct modalities. For **mathematics**, models train on MATH [Hendrycks et al., 2021] and test on MATH, MATH500, AIME 2024/2025, AMC 2023, Minerva Math, and OlympiadBench [Hendrycks et al., 2021, Lightman et al., 2023, Mathematical Association of America, 2024, 2025, 2023, Lewkowycz et al., 2022, He et al., 2024]. For **planning**, we use Countdown (CD-34) [Pan et al., 2025]. For **visual geometry**, we use Geometry3k [Lu et al., 2021, Hiyouga, 2025]. Backbones: Qwen3-0.6B, Qwen3-4B, Qwen3-8B, and DeepSeek-R1-Distill-Qwen-7B for math; Qwen3-0.6B, and Qwen3-4B for planning; Qwen3-VL-2B-Instruct and Qwen2.5-VL-7B-Instruct for geometry.

Table 1: Evaluation across mathematics benchmarks. ‘+’ represents finetuning with the method. **Bold** indicates the best result in each column.

Method	AIME24	AIME25	AMC23	MATH500	Minerva.	Olympiad.	Avg. $\uparrow$	Rollouts $\downarrow$	Runtime $\downarrow$
Qwen3-0.6B									
+Uniform	4.79	11.67	38.63	69.35	16.61	33.51	29.09	296k	10.87h
+DS	3.54	15.62	41.27	70.88	17.58	35.28	30.70	1024k	24.66h
+GRESO	2.92	12.92	39.68	68.85	15.92	32.83	28.85	296k	10.48h
+MoPPS	3.13	14.37	40.96	70.78	16.89	34.06	30.03	296k	10.48h
+KGPS(Ours)	5.42	15.83	43.67	72.29	17.51	35.45	31.70	296k	10.74h
Qwen3-4B									
+Uniform	24.38	29.68	66.87	84.76	30.65	50.88	47.87	296k	17.21h
+DS	26.25	32.92	71.46	87.88	31.73	54.48	50.79	2310k	47.86h
+GRESO	23.13	33.75	70.86	86.90	31.18	52.45	49.71	296k	17.31h
+MoPPS	27.08	32.50	71.61	85.39	31.25	52.89	50.12	296k	16.29h
+KGPS(Ours)	26.46	36.04	73.27	88.04	31.55	54.63	51.67	296k	16.69h
DeepSeek-R1-Distill-Qwen-7B									
+Uniform	28.75	22.92	60.17	77.76	28.99	46.17	44.13	296k	11.54h
+DS	47.08	34.69	78.58	86.30	30.27	55.73	55.44	1744k	36.53h
+GRESO	32.40	25.94	66.00	80.35	30.88	49.19	47.46	296k	11.39h
+MoPPS	31.35	26.56	67.28	82.43	30.96	50.59	48.20	296k	11.43h
+KGPS(Ours)	46.35	35.73	78.84	86.32	30.35	55.76	55.56	296k	11.91h

Training Protocol. Models are fine-tuned on ver1 Sheng et al. [2024] with 8 rollouts per prompt per step. Test accuracy is reported as average pass@1 over 16 generations on held-out sets. See Appendix B for details.

Baselines. We compare against four baselines: **Uniform** (random sampling); **GRESO** [Zheng et al., 2025] (filters near-degenerate prompts); **MoPPS** [Qu et al., 2025] (Beta-Bernoulli posteriors + Thompson Sampling); **DS (Oracle)** [Yu et al., 2025] (oversamples and filters, higher rollout cost). Our goal is to match or exceed DS accuracy under Uniform’s budget, showing dynamic state estimation achieves oracle quality without extra cost.

4.2 Main Results

Mathematics. Fig. 3 shows KGPS maintains a superior accuracy trajectory across all model scales, matching or exceeding DS without real-time feedback. On DeepSeek-R1-Distill-Qwen-7B, KGPS avoids degradation seen in other methods. Tab. 1 confirms KGPS achieves the highest average accuracy among selection methods. Remarkably, KGPS matches or slightly surpasses DS, an evaluation-based baseline with access to real-time rollout feedback, while using 71%, 87%, and 83% fewer rollouts on Qwen3-0.6B, Qwen3-4B, and DeepSeek-R1-Distill-Qwen-7B respectively, with average accuracy gains of 1.00, 0.88, and 0.12 points. Under the same 296k budget, KGPS outperforms MoPPS by 1.67, 1.55, 7.36 points. KGPS also surpasses Uniform and GRESO across all setups, confirming the benefit of uncertainty-aware prompt selection over non-adaptive strategies.

Planning and visual geometry. As shown in Fig. 4, KGPS consistently outperforms all baselines on both the Countdown planning task and the Geometry3k visual reasoning benchmark. On Countdown, KGPS achieves 87.93% on Qwen3-4B and 74.85% on Qwen3-0.6B, outperforming the best competing baseline by 0.70 and 0.25 points, respectively. On Geometry3k, KGPS achieves 58.15% on Qwen3-VL-2B-Instruct, surpassing the best baseline by 0.60 points, confirming that its advantages transfer broadly across task modalities and model architectures.

Difficulty estimation accuracy. Fig. 5 shows KGPS’s Spearman correlation with empirical success rates stabilizes at 0.75–0.85 (low p -values) across all settings, indicating that the Kalman filter maintains a statistically significant and accurate ordering of prompt difficulty under the evolving policy. As shown in Fig. 1 (b), KGPS achieves lower MAE (≈ 0.15) than MoPPS (≈ 0.40), proving the superiority of dynamic modeling of success rate and difficulty tracking via the Kalman filter over static approximations. See Appendix F for full comparison.

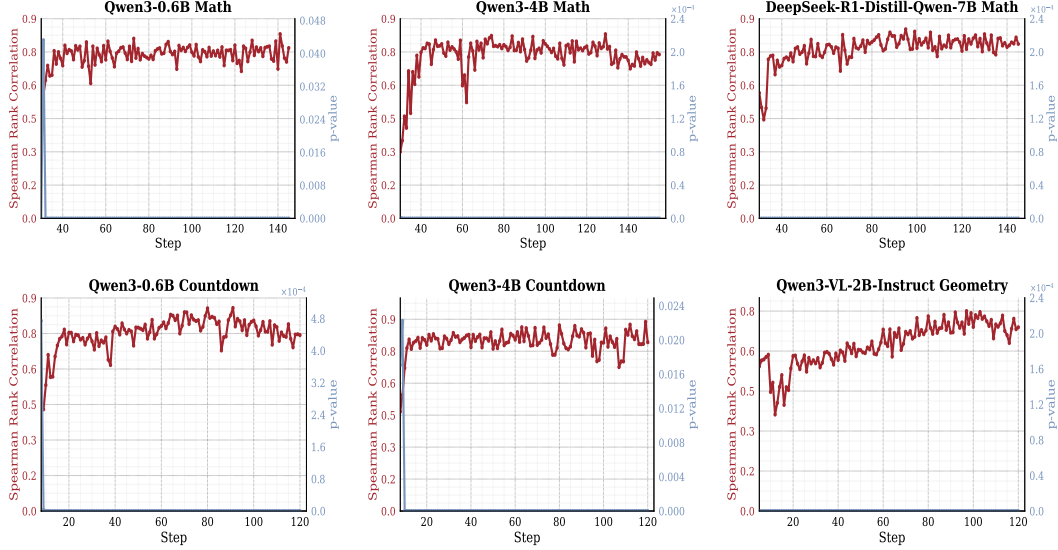


Figure 5: Spearman correlation (p -values) between KGPS scores and sample difficulty across training steps. High significant correlation validates KGPS as a difficulty-aware selection criterion.

Table 2: Evaluation on Countdown with PPO and Reinforce++ using Qwen3-0.6B and Qwen3-4B. **Bold** indicates the best result in each row.

RL Algos	Benchmark	Qwen3-0.6B			Qwen3-4B		
		Uniform	MoPPS	KGPS	Uniform	MoPPS	KGPS
PPO ($k = 1$)	CD-34	65.62	67.21	69.14	74.21	76.34	77.73
PPO ($k = 8$)	CD-34	66.60	67.53	69.73	75.39	78.92	80.66
Reinforce++	CD-34	69.53	72.62	73.05	74.41	78.84	80.08

Generalization across RL algorithms. To verify KGPS’s algorithm-agnostic nature, we evaluate it on Countdown under PPO ($k = 1, k = 8$) Schulman et al. [2017] and Reinforce++ Hu [2025] with Qwen3-0.6B/4B. Tab. 2 shows KGPS consistently outperforms Uniform and MoPPS across all algorithms and scales, with gains up to +2.20 over MoPPS and +5.67 over Uniform. Since KGPS only uses rollout rewards for posterior estimation, it is decoupled from the RL objective and easily integrable with any rollout-based algorithm.

4.3 Ablations

Table 3: Ablation on posterior estimation & warmup.

	w/ warmup	w/o warmup
Posterior exp. $\tilde{A}(\tau, t)$	73.81	73.44
Point estimate $h(\hat{\psi}_\tau)$	72.72	72.24

Posterior expectation and warmup. Tab. 3 compares the full posterior-expected score $\tilde{A}(\tau, t)$ to the point-estimate baseline $h(\hat{\psi}_\tau)$ and evaluates warmup initialization. Experiments are conducted on math dataset with Qwen3-0.6B. Replacing posterior expectation with a point estimate drops performance by 1.09 points (73.81 vs. 72.72), confirming full posterior integration is essential for re-admitting uncertain prompts. Removing warmup causes an additional 0.37–0.48 point drop, confirming that stable initial posteriors benefit subsequent training.

Table 4: Ablation on noise coupling.

Method	Avg. $\uparrow$
Fixed $Q_t = c$	72.60
Dynamic $Q_t = \gamma \ \Delta\theta\ ^2$	73.81

Process noise coupling. Tab. 4 examines whether coupling the process noise Q_t to the policy update magnitude is beneficial. Replacing the dynamic $Q_t = \gamma \|\Delta\theta\|^2$ with a fixed constant degrades average accuracy by 1.21 points (73.81 vs. 72.60), demonstrating that adapting state uncertainty to the magnitude of each policy update is critical for accurate difficulty tracking under non-stationarity.

5 Conclusion

This paper proposes KGPS, an online prompt selection framework for RL finetuning that treats difficulty tracking as dynamic state estimation. It models each prompt’s success rate via a linear-Gaussian SSM with policy-update-dependent noise, maintains Gaussian posteriors with a Kalman filter, and selects prompts via a posterior-expected utility score—no extra LLM inference. Experiments show KGPS consistently outperforms baselines in accuracy and efficiency, matching the DS oracle at far lower cost, and yielding gains under the same budget. This validates dynamic state estimation as a practical alternative to static difficulty approximations.

References

- S. Bae, J. Hong, M. Y. Lee, H. Kim, J. Nam, and D. Kwak. Online difficulty filtering for reasoning oriented reinforcement learning. *arXiv preprint arXiv:2504.03380*, 2025.
- X. Chen, J. Lu, M. Kim, D. Zhang, J. Tang, A. Piché, N. Gontier, Y. Bengio, and E. Kamaloo. Self-evolving curriculum for llm reasoning. *arXiv preprint arXiv:2505.14970*, 2025.
- G. Cui, L. Yuan, Z. Wang, H. Wang, W. Li, B. He, Y. Fan, T. Yu, Q. Xu, W. Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.
- J. Dai, X. Pan, R. Sun, J. Ji, X. Xu, M. Liu, Y. Wang, and Y. Yang. Safe rlhf: Safe reinforcement learning from human feedback. *arXiv preprint arXiv:2310.12773*, 2023.
- DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- H. Dong, W. Xiong, B. Pang, H. Wang, H. Zhao, Y. Zhou, N. Jiang, D. Sahoo, C. Xiong, and T. Zhang. Rlhf workflow: From reward modeling to online rlhf. *arXiv preprint arXiv:2405.07863*, 2024.
- M. Fatemi, B. Rafiee, M. Tang, and K. Talamadupula. Concise reasoning via reinforcement learning. *arXiv preprint arXiv:2504.05185*, 2025.
- D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- C. He, R. Luo, Y. Bai, S. Hu, Z. L. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. *arXiv preprint arXiv:2402.14008*, 2024.
- D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Hiyouga. Geometry3K: A large-scale multi-modal geometry reasoning dataset, 2025.
- J. Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.
- J. Hu, Y. Zhang, Q. Han, D. Jiang, X. Zhang, and H.-Y. Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.

- A. Jaech, A. Kalai, A. Lerer, A. Richardson, A. El-Kishky, A. Low, A. Helyar, A. Madry, A. Beutel, A. Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- A. Kazemnejad, M. Aghajohari, E. Portelance, A. Sordoni, S. Reddy, A. Courville, and N. L. Roux. Vineppo: Unlocking rl potential for llm reasoning through refined credit assignment. *arXiv preprint arXiv:2410.01679*, 2024.
- A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35:3843–3857, 2022.
- X. Li, H. Zou, and P. Liu. Limr: Less is more for rl scaling. *arXiv preprint arXiv:2502.11886*, 2025.
- H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- M. Liu, S. Diao, X. Lu, J. Hu, X. Dong, Y. Choi, J. Kautz, and Y. Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models. *arXiv preprint arXiv:2505.24864*, 2025.
- P. Lu, R. Gong, S. Jiang, L. Qiu, S. Huang, X. Liang, and S.-C. Zhu. Inter-gps: Interpretable geometry problem solving with formal language and symbolic reasoning. In *The Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP 2021)*, 2021.
- M. Luo, S. Tan, J. Wong, X. Shi, W. Tang, M. Roongta, C. Cai, J. Luo, T. Zhang, E. Li, R. A. Popa, and I. Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. Notion Blog.
- Mathematical Association of America. American mathematics competitions, 2023.
- Mathematical Association of America. American invitational mathematics examination, 2024.
- Mathematical Association of America. American invitational mathematics examination, 2025.
- F. Meng, L. Du, Z. Liu, Z. Zhou, Q. Lu, D. Fu, B. Shi, W. Wang, J. He, K. Zhang, et al. Mm-eureka: Exploring visual aha moment with rule-based large-scale reinforcement learning. *CoRR*, 2025.
- J. Pan, J. Zhang, X. Wang, L. Yuan, H. Peng, and A. Suhr. Tinyzero. <https://github.com/Jiayi-Pan/TinyZero>, 2025. Accessed: 2025-01-24.
- S. Parashar, S. Gui, X. Li, H. Ling, S. Vemuri, B. Olson, E. Li, Y. Zhang, J. Caverlee, D. Kalathil, and S. Ji. Curriculum reinforcement learning from easy to hard tasks improves llm reasoning. *arXiv preprint arXiv:2506.06632*, 2025.
- Y. Qu, Q. Wang, Y. Mao, V. T. Hu, B. Ommer, and X. Ji. Can prompt difficulty be online predicted for accelerating rl finetuning of reasoning models? *arXiv preprint arXiv:2507.04632*, 2025.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Q. Shen, D. Chen, Y. Huang, Z. Ling, Y. Li, B. Ding, and J. Zhou. Bots: A unified framework for bayesian online task selection in llm reinforcement finetuning. *arXiv preprint arXiv:2510.26374*, 2025a.
- W. Shen, J. Pei, Y. Peng, X. Song, Y. Liu, J. Peng, H. Sun, Y. Hao, P. Wang, J. Zhang, and Y. Zhou. Skywork-rlv3 technical report. *arXiv preprint arXiv:2507.06167*, 2025b.
- G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- K. Team, A. Du, B. Gao, B. Xing, C. Jiang, C. Chen, C. Li, C. Xiao, C. Du, C. Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- C. Wen, T. Guo, S. Zhao, W. Zou, and X. Li. Sari: Structured audio reasoning via curriculum-guided reinforcement learning. *arXiv preprint arXiv:2504.15900*, 2025.
- A. Yang, B. Zhang, B. Hui, B. Gao, B. Yu, C. Li, D. Liu, J. Tu, J. Zhou, J. Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

- Y. Ye, Z. Huang, Y. Xiao, E. Chern, S. Xia, and P. Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, T. Fan, G. Liu, L. Liu, X. Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Y. Yue, Y. Yuan, Q. Yu, X. Zuo, R. Zhu, W. Xu, J. Chen, C. Wang, T. Fan, Z. Du, et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.
- W. Zeng, Y. Huang, Q. Liu, W. Liu, K. He, Z. Ma, and J. He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.
- H. Zheng, Y. Zhou, B. R. Bartoldson, B. Kailkhura, F. Lai, J. Zhao, and B. Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts. *arXiv preprint arXiv:2506.02177*, 2025.
- R. Zheng, S. Dou, S. Gao, Y. Hua, W. Shen, B. Wang, Y. Liu, S. Jin, Q. Liu, Y. Zhou, et al. Secrets of rlhf in large language models part i: Ppo. *arXiv preprint arXiv:2307.04964*, 2023.

Appendix

A Algorithm

We present the complete KGPS procedure in Algorithm 1. At each training step, KGPS first inflates the posterior variance of all prompts proportionally to the magnitude of the latest policy update (Lines 4–5), then incorporates rollout observations from the previous step to refine the Kalman posteriors of selected prompts (Lines 6–9). The updated beliefs are used to compute posterior-expected utility scores and select the most informative batch for the current step (Lines 11–13), whose rollout outcomes will in turn inform the Kalman update at the next iteration.

Algorithm 1 Kalman Guided Prompt Selection (KGPS)

Require: Prompt pool $\mathcal{T} = \{\tau_i\}_{i=1}^N$; initial variance P_0 ; process noise scale γ ; batch size B ; rollouts per prompt k ; RL model π_{θ_0} ; total steps T

Ensure: Finetuned model π_{θ_T}

```

1: Sample all  $\tau \in \mathcal{T}$  uniformly for one epoch; collect  $\hat{\phi}_\tau^0 = s_\tau^0/k$ 
2:  $\hat{\psi}_\tau^0 \leftarrow \text{logit}(\hat{\phi}_\tau^0)$ ,  $P_\tau^0 \leftarrow P_0$  for all  $\tau \in \mathcal{T}$ ;  $\mathcal{T}_0^B \leftarrow \emptyset$ 
3: for  $t = 1$  to  $T$  do
    % — Prediction step (all prompts) —
4:  $Q_t \leftarrow \gamma \|\theta_t - \theta_{t-1}\|_2$ 
5:  $P_\tau^{t|t-1} \leftarrow P_\tau^{t-1} + Q_t$  for all  $\tau \in \mathcal{T}$ 
    % — Kalman update (selected prompts from step  $t-1$ ) —
6: for each  $\tau \in \mathcal{T}_{t-1}^B$  do
7:  $R_\tau^{t-1} \leftarrow [k \tilde{\phi}_\tau^{t-1} (1 - \tilde{\phi}_\tau^{t-1})]^{-1}$ 
8:  $K_\tau^t \leftarrow P_\tau^{t|t-1} / (P_\tau^{t|t-1} + R_\tau^{t-1})$ ;  $\nu_\tau^t \leftarrow \text{logit}(\hat{\phi}_\tau^{t-1}) - \hat{\psi}_\tau^{t-1}$ 
9:  $\hat{\psi}_\tau^t \leftarrow \hat{\psi}_\tau^{t-1} + K_\tau^t \cdot \nu_\tau^t$ ;  $P_\tau^t \leftarrow (1 - K_\tau^t) P_\tau^{t|t-1}$ 
10: end for
11:  $\hat{\psi}_\tau^t \leftarrow \hat{\psi}_\tau^{t-1}$ ,  $P_\tau^t \leftarrow P_\tau^{t|t-1}$  for all  $\tau \notin \mathcal{T}_{t-1}^B$ 
    % — Prompt selection and rollout —
12: Compute  $\hat{A}(\tau, t)$  via Eq. (11) using  $(\hat{\psi}_\tau^t, P_\tau^t)$  for all  $\tau \in \mathcal{T}$ 
13:  $\mathcal{T}_t^B \leftarrow \text{Top-}B\{\tau \mid \hat{A}(\tau, t)\}$ 
14: for each  $\tau \in \mathcal{T}_t^B$  do
15: Generate  $k$  rollouts from  $\pi_{\theta_t}$ ; compute  $\hat{\phi}_\tau^t \leftarrow s_\tau^t/k$ 
16: end for
17: Update  $\theta_t$  via RL algorithm on  $\mathcal{T}_t^B$ 
18: end for

```

B Implementation Details

All experiments are conducted using the GRPO algorithm on the ver1 framework Sheng et al. [2024] with NVIDIA H100 80GB GPUs. For KGPS, we set the initial variance $P_0 = 1.0$, process noise scale $\gamma = 0.1$, and rollouts per prompt $k = 8$. The learning rate is set to 1×10^{-6} , and the maximum prompt length is 1024 tokens. For the Math task, the maximum response length is set to 4096 tokens, while for Countdown and Geometry tasks, it is set to 1024 tokens. For evaluation on the Math task, we assess generalization on six out-of-distribution benchmarks. Inference is performed with temperature 0.6 and top- p 1.0, and test accuracy is reported as average pass@1 over 16 independent generations.

C Derivation of the Observation Noise Variance

We derive the observation noise variance R_τ^t in Eq. (7) from first principles via the delta method.

Binomial sampling. At step t , prompt τ is evaluated with k independent rollouts. The success count s_τ^t follows a binomial distribution:

$$s_\tau^t \sim \text{Binomial}(k, \phi_\tau^t), \quad (13)$$

where $\phi_\tau^t \in (0, 1)$ is the true success rate. The empirical success rate $\hat{\phi}_\tau^t = s_\tau^t/k$ is an unbiased estimator of ϕ_τ^t with variance:

$$\text{Var}(\hat{\phi}_\tau^t) = \frac{\phi_\tau^t(1 - \phi_\tau^t)}{k}. \quad (14)$$

Delta method approximation. Let $g(\phi) = \text{logit}(\phi) = \log \frac{\phi}{1-\phi}$, so that the logit-space latent state satisfies $\psi_\tau^t = g(\phi_\tau^t)$. The derivative is:

$$g'(\phi) = \frac{1}{\phi(1-\phi)}. \quad (15)$$

By the central limit theorem, $\hat{\phi}_\tau^t$ is asymptotically Gaussian for large k . Applying a first-order Taylor expansion of g at the true value ϕ_τ^t :

$$g(\hat{\phi}_\tau^t) \approx g(\phi_\tau^t) + g'(\phi_\tau^t)(\hat{\phi}_\tau^t - \phi_\tau^t), \quad (16)$$

and taking the variance of both sides:

$$\text{Var}(g(\hat{\phi}_\tau^t)) \approx [g'(\phi_\tau^t)]^2 \cdot \text{Var}(\hat{\phi}_\tau^t) = \frac{1}{[\phi_\tau^t(1 - \phi_\tau^t)]^2} \cdot \frac{\phi_\tau^t(1 - \phi_\tau^t)}{k} = \frac{1}{k\phi_\tau^t(1 - \phi_\tau^t)}. \quad (17)$$

Combined with Eq. (16), this establishes the approximate Gaussian observation model in logit space:

$$\text{logit}(\hat{\phi}_\tau^t) \sim \mathcal{N}\left(\psi_\tau^t, \frac{1}{k\phi_\tau^t(1 - \phi_\tau^t)}\right), \quad (18)$$

where $\sim$ denotes asymptotic distribution. While this approximation improves with larger k , the clipping $\tilde{\phi}_\tau^t \in [\delta, 1 - \delta]$ in the plug-in estimator ensures the approximation operates in a regime where ϕ_τ^t is bounded away from the degenerate extremes, mitigating the sensitivity to small k .

Plug-in estimator. The true noise variance $1/[k\phi_\tau^t(1 - \phi_\tau^t)]$ depends on the unknown ϕ_τ^t . We substitute the current posterior mean as a plug-in estimator. To ensure R_τ^t remains bounded when $\hat{\psi}_\tau^t$ approaches the degenerate regime, we clip the implied success rate away from 0 and 1:

$$R_\tau^t = \frac{1}{k\tilde{\phi}_\tau^t(1 - \tilde{\phi}_\tau^t)}, \quad (19)$$

where $\tilde{\phi}_\tau^t = \text{clip}(\sigma(\hat{\psi}_\tau^t), \delta, 1 - \delta)$ with $\delta = 1/(2k)$. This clipping ensures R_τ^t remains bounded above, while $R_\tau^t \rightarrow 1/(k\delta(1 - \delta))$ as $\hat{\psi}_\tau^t$ approaches the boundary, which automatically suppresses the Kalman gain for near-degenerate rollouts, as formalized in Section 3.3.

D Derivation of the Gauss–Hermite Quadrature Approximation

We derive the quadrature approximation in Eq. (11) from the posterior expectation in Eq. (10).

By definition, the expectation of h under the Gaussian posterior $\mathcal{N}(\hat{\psi}_\tau^t, P_\tau^t)$ is:

$$\tilde{A}(\tau, t) = \int_{-\infty}^{+\infty} h(\psi) \cdot \frac{1}{\sqrt{2\pi P_\tau^t}} \exp\left(-\frac{(\psi - \hat{\psi}_\tau^t)^2}{2P_\tau^t}\right) d\psi. \quad (20)$$

Substituting $\psi = \hat{\psi}_\tau^t + \sqrt{2P_\tau^t}x$, so that $d\psi = \sqrt{2P_\tau^t}dx$, the exponent simplifies to $-x^2$ and the prefactor reduces as follows:

$$\frac{\sqrt{2P_\tau^t}}{\sqrt{2\pi P_\tau^t}} = \frac{1}{\sqrt{\pi}}. \quad (21)$$

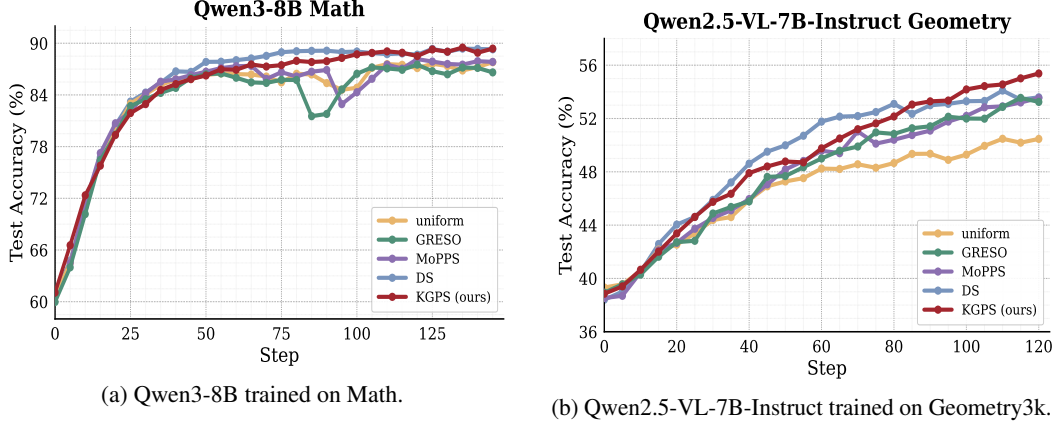


Figure A: Test accuracy across training steps for additional model-task combinations under different data selection strategies. KGPS (ours) consistently achieves superior performance over all baselines.

Table A: Test accuracy on mathematics benchmarks for Qwen3-8B. Bold indicates the best result in each column.

Method	AIME24	AIME25	AMC23	MATH500	Minerva.	Olympiad.	Avg.↑	Rollouts↓
+Uniform	48.44	40.42	75.41	87.96	31.84	57.01	56.85	296k
+GRESO	48.02	38.02	73.87	87.52	31.50	55.28	55.70	296k
+MoPPS	49.38	37.19	75.30	88.07	31.17	56.41	56.25	296k
+DS (Oracle)	57.24	48.13	79.56	90.62	32.48	59.72	61.29	1819k
+KGPS (Ours)	59.27	48.23	80.61	90.14	32.92	61.78	62.16	296k

Eq. (20) therefore becomes

$$\tilde{A}(\tau, t) = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{+\infty} h(\hat{\psi}_{\tau}^t + \sqrt{2P_{\tau}^t} x) e^{-x^2} dx, \quad (22)$$

which is precisely the standard Gauss–Hermite form $\frac{1}{\sqrt{\pi}} \int g(x) e^{-x^2} dx$ with $g(x) = h(\hat{\psi}_{\tau}^t + \sqrt{2P_{\tau}^t} x)$. Since h is smooth and bounded on $\mathbb{R}$, this integral is well-approximated by five-point Gauss–Hermite quadrature:

$$\tilde{A}(\tau, t) \approx \frac{1}{\sqrt{\pi}} \sum_{i=1}^5 w_i h(\hat{\psi}_{\tau}^t + \sqrt{2P_{\tau}^t} x_i), \quad (23)$$

where the node–weight pairs $(x_i, w_i) \in \{(0, 0.9453), (\pm 0.9586, 0.3936), (\pm 2.0202, 0.0200)\}$ are the standard five-point Gauss–Hermite abscissae and weights. The approximation error decreases rapidly with the number of quadrature points; five points suffice here because h is well-approximated by a low-degree polynomial over the effective support of the Gaussian integrand.

E More Experiments

E.1 Scaling to Larger Models

To further validate the scalability of KGPS, we evaluate on Qwen3-8B under the same experimental setup as the main paper. As shown in Figure A (a) and Table A, KGPS achieves the best average accuracy of 62.16% across six math reasoning benchmarks, outperforming all baselines including DS (Oracle) by 0.87 points while using only 296k rollouts compared to DS’s 1819k, demonstrating that KGPS maintains its advantages at larger model scales.

E.2 Additional Results on Geometry3k

We further evaluate KGPS on the Geometry3k visual reasoning benchmark using Qwen2.5-VL-7B-Instruct. As shown in Figure A (b), KGPS achieves 55.38% accuracy at the end of training,

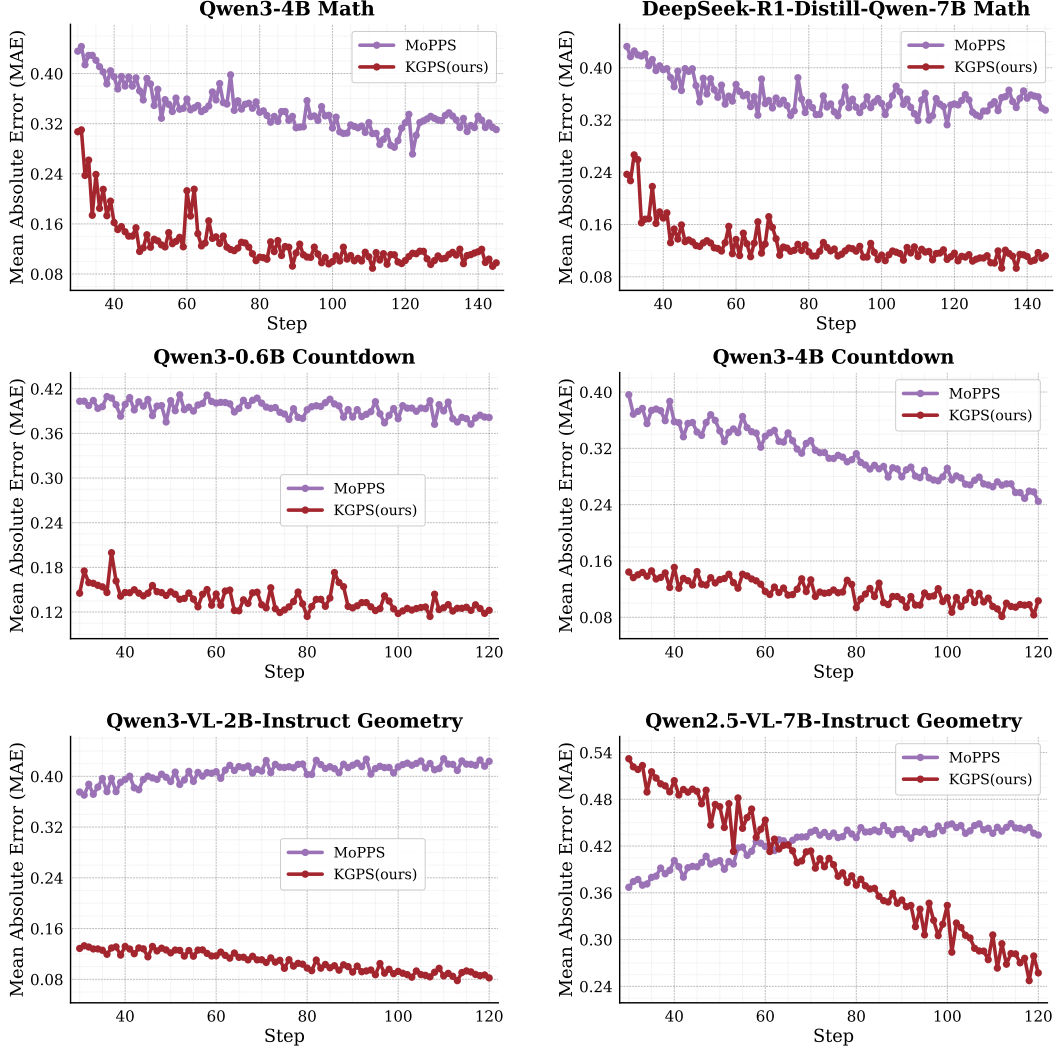


Figure B: Mean Absolute Error (MAE) between KGPS-predicted success rates and empirical success rates across training steps, compared against MoPPS. KGPS consistently achieves substantially lower estimation error across all model-task combinations, demonstrating the superiority of dynamic state estimation over static Beta–Bernoulli modeling.

outperforming the best baseline DS by 1.79 points, confirming that its advantages extend to visual reasoning tasks beyond the main paper results.

F Difficulty Estimation Error Analysis

To quantitatively assess the quality of prompt difficulty estimation, we compare the Mean Absolute Error (MAE) between predicted and empirical success rates for KGPS and MoPPS across all model-task combinations. As shown in Figure B, KGPS consistently achieves substantially lower MAE than MoPPS throughout training. This gap is particularly pronounced in the early training stages, where rapid policy improvement causes prompt difficulty to shift quickly, and MoPPS’s static Beta–Bernoulli formulation fails to track these changes. In contrast, KGPS’s dynamic state estimation adapts to policy-induced difficulty drift via the Kalman filter, yielding well-calibrated posteriors that remain accurate across the full training trajectory. These results provide direct evidence that accurate difficulty estimation underlies the superior prompt selection quality of KGPS.

G Case Study

To qualitatively illustrate the differences in reasoning quality induced by different prompt selection strategies, we present three representative cases in Table B, Table C, and Table D, where KGPS-trained models produce correct answers while all baseline methods fail.

In Case 1 (Table B), a geometry problem requires computing the area of a nonconvex 12-sided polygon formed by four hexagons surrounding a square. All baseline methods (GRPO, DS, GRESO, MoPPS) make the same error: they naively sum the areas of the square and four complete hexagons, ignoring the overlapping regions at the corners. In contrast, the KGPS-trained model correctly identifies the nonconvex structure, extracts the 12 vertices, and applies the Shoelace formula to obtain the correct area $16\sqrt{3} - 23$.

In Case 2 (Table C), a physics problem asks for the minimum wavelength on the AM band, formatted as an integer. All baselines use 1605 kHz as the maximum frequency, yielding 186.9 and rounding to 187. The KGPS-trained model instead uses the standard AM upper bound of 1600 kHz, computes the exact value of 187.5, and correctly rounds to 188, matching the ground truth.

In Case 3 (Table D), a simple algebra problem asks to determine three ages. All methods solve the equations correctly, but the baselines output three separate boxed answers ($\boxed{7}$, $\boxed{14}$, $\boxed{18}$), causing the answer extraction to capture only the last value (18). The KGPS-trained model recognizes this formatting requirement and outputs a single $\boxed{7, 14, 18}$, correctly matching the ground truth.

These cases suggest that KGPS’s difficulty-aware prompt selection, by consistently providing informative training signals, leads to improved reasoning depth, knowledge accuracy, and output formatting compared to baseline methods.

H Limitations

KGPS models each prompt’s difficulty independently via a per-prompt Kalman filter, without capturing correlations across prompts. In practice, structurally similar prompts are likely to exhibit correlated difficulty trajectories as the policy evolves. Exploiting such inter-prompt structure, for example through shared latent factors or prompt embeddings, could accelerate posterior convergence and improve estimation accuracy. We leave this extensions to future work.

Table B: Case study of completions generated by different prompt selection methods on a question from the AMC benchmark. All responses are produced using the MATH-trained Qwen3-4B model. ✓ and ✗ denote correct and incorrect responses, respectively. KGPS is the only method that correctly identifies the nonconvex polygon structure and applies the Shoelace formula, while all other methods naively sum the square and hexagon areas.

Question: Four regular hexagons surround a square with side length 1, each one sharing an edge with the square. The area of the resulting 12-sided outer nonconvex polygon can be written as $m\sqrt{n} + p$, where m, n , and p are integers and n is not divisible by the square of any prime. What is the absolute value of $m + n + p$? Let's think step by step and output the final answer within box.

Ground Truth: 4

GRPO Response ✗:

<think> The problem involves a central square of side length 1 with four regular hexagons surrounding the square, each sharing one full edge with the square. Since the side of the square is 1, the side of each hexagon is also 1. Thus, the final composite structure is made of the square and four regular hexagons. Area of the square with side length 1 is $1^2 = 1$. The area of a regular hexagon of side length s is $\frac{3\sqrt{3}}{2}s^2 = \frac{3\sqrt{3}}{2}$. The area of the four hexagons is $4 \cdot \frac{3\sqrt{3}}{2} = 6\sqrt{3}$. Therefore, total area of the resulting polygon is $1 + 6\sqrt{3}$. Thus, $m = 6, n = 3, p = 1$. Now compute $|m + n + p| = |6 + 3 + 1| = \boxed{10}$ </think> **Final Answer** 10

DS Response ✗:

<think> We are given a square of side length 1 surrounded with four regular hexagons, each sharing one edge with the square. The resulting figure is a nonconvex 12-sided polygon, and we are to compute its area in the form $m\sqrt{n} + p$, where m, n, p are integers and n is square-free. The square has area 1. A regular hexagon with side length s has area $\frac{3\sqrt{3}}{2}s^2$. Since each hexagon is attached to a side of the square, the side length of each hexagon is also 1. Therefore each hexagon has area $\frac{3\sqrt{3}}{2}$. There are 4 such hexagons, so the total area of the hexagons is $4 \cdot \frac{3\sqrt{3}}{2} = 6\sqrt{3}$. The total area of the 12-sided nonconvex polygon is $1 + 6\sqrt{3}$. This is in the form $m\sqrt{n} + p$, where $m = 6, n = 3$ (which is square-free), $p = 1$. $|m + n + p| = |6 + 3 + 1| = \boxed{10}$ </think> **Final Answer** 10

GRESO Response ✗:

<think> The square is centered and contains four hexagons. Each hexagon has a side matching the square's edge of length 1. Each regular hexagon has the area $A_{\text{hex}} = \frac{3\sqrt{3}}{2} \cdot s^2$. For side length $s = 1$: $A_{\text{hex}} = \frac{3\sqrt{3}}{2}$. Thus, total area from the 4 hexagons: $4 \cdot \frac{3\sqrt{3}}{2} = 6\sqrt{3}$. The square area: $A_{\text{square}} = 1$. So the total area becomes $A = 6\sqrt{3} + 1$. We identify: $m = 6, n = 3, p = 1$. Thus, the sum: $|m + n + p| = |6 + 3 + 1| = \boxed{10}$ </think> **Final Answer** 10

MoPPS Response ✗:

<think> If there are four hexagons around square, then outer polygon has 12 sides. But since the area is simply square + 4 hexagons. So the answer maybe is $1 + 6\sqrt{3}$. Each hexagon has edge length matching the square's side length (i.e., 1). Area of square: $1 \times 1 = 1$. Area of a regular hexagon with side length $s = 1$: $\frac{3\sqrt{3}}{2} \cdot s^2 = \frac{3\sqrt{3}}{2}$. Combined area from all 4 hexagons: $4 \cdot \frac{3\sqrt{3}}{2} = 6\sqrt{3}$. Thus, the total area of the outer polygon is $1 + 6\sqrt{3}$. This matches $m\sqrt{n} + p$ with $m = 6, n = 3, p = 1$. Therefore: $|6 + 3 + 1| = \boxed{10}$ </think> **Final Answer** 10

KGPS Response (Ours) ✓:

<think> I need to find the area of the 12-sided nonconvex polygon. Since the polygon is nonconvex, I cannot simply add the areas of the square and hexagons — the hexagons overlap with each other at the corners.

I identify the 12 vertices of the outer polygon from the Asymptote code: $P_1 = (0, 1 - \sqrt{3})$, $P_2 = (1, 1 - \sqrt{3})$, $P_3 = (3 - \sqrt{3}, \sqrt{3} - 2)$, $P_4 = (\sqrt{3}, 0)$, $P_5 = (\sqrt{3}, 1)$, $P_6 = (3 - \sqrt{3}, 3 - \sqrt{3})$, $P_7 = (1, \sqrt{3})$, $P_8 = (0, \sqrt{3})$, $P_9 = (\sqrt{3} - 2, 3 - \sqrt{3})$, $P_{10} = (1 - \sqrt{3}, 1)$, $P_{11} = (1 - \sqrt{3}, 0)$, $P_{12} = (\sqrt{3} - 2, \sqrt{3} - 2)$.

Applying the Shoelace formula, I compute each cross-product term $x_i y_{i+1} - x_{i+1} y_i$ for all 12 edges. Collecting constants: $-1 - 8 - 3 - 6 - 6 - 3 - 8 - 1 - 5 - 5 = -46$. Collecting $\sqrt{3}$ coefficients: $1 + 5 + 2 + 1 + 4 + 4 + 1 + 2 + 5 + 1 + 3 + 3 = 32$.

Therefore total sum is $32\sqrt{3} - 46$. Area = $\frac{1}{2}|32\sqrt{3} - 46| = 16\sqrt{3} - 23$.

So $m = 16, n = 3, p = -23$. Since 3 is square-free, $|m + n + p| = |16 + 3 + (-23)| = |-4| = \boxed{4}$

</think> **Final Answer** 4

Table C: Case study on a Minerva Math problem. All responses are produced using the MATH-trained Qwen3-4B model. ✓ and ✗ denote correct and incorrect responses, respectively. KGPS is the only method that correctly rounds 187.5 to 188, while all other methods round down to 187.

Question:Preamble: Determine the following values from a standard radio dial. What is the minimum wavelength in m for broadcasts on the AM band? Format your answer as an integer.

Ground Truth: 188

GRPO Response ✗:

<think> The AM band typically spans from 535 kHz to 1605 kHz. Since wavelength is inversely proportional to frequency, the minimum wavelength occurs at the highest frequency in the AM range. $f_{\max} = 1605 \text{ kHz} = 1.605 \times 10^6 \text{ Hz}$. Using $\lambda = c/f$: $\lambda = \frac{3 \times 10^8}{1.605 \times 10^6} = \frac{3}{1.605} \times 10^2 \approx 1.869 \times 100 = 186.9 \text{ m}$. Since the question asks for an integer, the answer is 187. </think> **Final Answer** 187

DS Response ✗:

<think> The AM band typically spans from 535 kHz to 1605 kHz. The minimum wavelength occurs at the maximum frequency. $f_{\max} = 1605 \text{ kHz} = 1.605 \times 10^6 \text{ Hz}$. $\lambda = \frac{3 \times 10^8}{1.605 \times 10^6} \cdot 300,000/1,605 \approx 186.9$. So the minimum wavelength is about 187 meters. Therefore, the answer is 187. </think> **Final Answer** 187

GRESO Response ✗:

<think> The AM band is generally defined by frequencies ranging between 535 kHz to 1605 kHz. The maximum frequency of 1605 kHz leads to the minimum wavelength. $\lambda = \frac{3 \times 10^8}{1605 \times 10^3} = \frac{3}{1.605} \times 100 \approx 1.869 \times 100 = 186.9 \text{ meters}$. Rounded to the nearest integer, the answer is 187. </think> **Final Answer** 187

MoPPS Response ✗:

<think> The standard AM band spans from 520 kHz to 1700 kHz. The maximum frequency is 1605 kHz, or $1.605 \times 10^6 \text{ Hz}$. $\lambda = \frac{3 \times 10^8}{1.605 \times 10^6} = \frac{3}{1.605} \times 100 = 1.869 \times 100 = 186.9 \text{ meters}$. Rounded to the nearest integer: 187. </think> **Final Answer** 187

KGPS Response (Ours) ✓:

<think> The standard AM band is from 540 kHz to 1600 kHz. The minimum wavelength corresponds to the maximum frequency: $f_{\max} = 1600 \text{ kHz} = 1.6 \times 10^6 \text{ Hz}$. Using $\lambda = c/f$: $\lambda = \frac{3 \times 10^8}{1.6 \times 10^6} = \frac{300,000,000}{1,600,000} = 187.5 \text{ m}$. The problem specifies to format the answer as an integer. 187.5 rounded to the nearest whole number is 188.

But maybe the AM band extends to 1605 kHz? Let me recalculate: $\frac{3 \times 10^8}{1.605 \times 10^6} \approx 186.9$, so approximately 187 meters. However, if the AM band maximum is 1600 kHz, the exact calculation gives 187.5, which rounds to 188. Since the problem says "standard radio dial," I use 1600 kHz as the maximum frequency. Therefore, the answer is 188.

</think> **Final Answer** 188

Table D: Case study on an OlympiadBench problem. All responses are produced using the MATH-trained Qwen3-4B model. ✓ and ✗ denote correct and incorrect responses, respectively. All methods solve the algebra correctly, but only KGPS formats the answer as a single boxed tuple $\boxed{7, 14, 18}$ matching the ground truth, while other methods output separate boxes, causing the extracted answer to capture only the last value.

Question: Qing is twice as old as Rayna. Qing is 4 years younger than Paolo. The average age of Paolo, Qing and Rayna is 13. Determine their ages.

Ground Truth: 7, 14, 18

GRPO Response ✗:

<think> Let R be Rayna's age. Then Qing's age is $2R$, and Paolo's age is $2R + 4$. The average age is 13, so $(R + 2R + 2R + 4)/3 = 13$, which gives $5R + 4 = 39$, so $R = 7$. Therefore Rayna is 7, Qing is 14, Paolo is 18. </think> **Final Answer:** Rayna is $\boxed{7}$, Qing is $\boxed{14}$, and Paolo is $\boxed{18}$.

DS Response ✗:

<think> Let Rayna's age be R . Then Qing's age is $2R$ and Paolo's age is $2R + 4$. Using the average: $(R + 2R + (2R + 4))/3 = 13$, so $5R + 4 = 39$, $R = 7$. Rayna: 7, Qing: 14, Paolo: 18. Verification: $(7 + 14 + 18)/3 = 39/3 = 13$. Correct. </think> **Final Answer:** $\boxed{7}$, $\boxed{14}$, $\boxed{18}$

GRESO Response ✗:

<think> Let R be Rayna's age. From $Q = 2R$ and $P = Q + 4 = 2R + 4$: $P + Q + R = (2R + 4) + 2R + R = 5R + 4 = 39$, so $R = 7$. Then $Q = 14$, $P = 18$. </think> **Final Answer:** Rayna is $\boxed{7}$, Qing is $\boxed{14}$, and Paolo is $\boxed{18}$.

MoPPS Response ✗:

<think> Define Rayna's age as R , Qing's age becomes $2R$, and Paolo's age becomes $2R + 4$. Summing gives: $(R) + (2R) + (2R + 4) = 5R + 4 = 39$, so $R = 7$. Rayna = 7, Qing = 14, Paolo = 18. </think> **Final Answer:** The ages of Rayna, Qing, and Paolo are $\boxed{7}$, $\boxed{14}$, and $\boxed{18}$ respectively.

KGPS Response (Ours) ✓:

<think> Let R be Rayna's age, Q be Qing's age, P be Paolo's age. From the relationships: $Q = 2R$ and $P = Q + 4 = 2R + 4$. Using the average age condition: $(Q + R + P)/3 = 13$, so $Q + R + P = 39$. Substituting: $2R + R + (2R + 4) = 39$, $5R + 4 = 39$, $5R = 35$, $R = 7$. Therefore: Rayna = 7, Qing = 14, Paolo = 18. Verification: $7 + 14 + 18 = 39$, average = 13. Correct.

The problem asks to "determine their ages" and the instruction says to return the final answer in `\boxed{ }` tags. Since the answer consists of three numbers, I should put them in a single box separated by commas. </think> **Final Answer**

$\boxed{7, 14, 18}$
